Progressive Retrieval Attention with AirLLM: Independent Weight and Semantic-Context Streaming

Jorge Simão

August 2026

Abstract

AirLLM makes models larger than accelerator memory executable by loading one decoder layer, or selected experts, immediately before use and releasing its weights afterward. Progressive Retrieval Attention (PRA) addresses a different state variable: it routes over persistent context and materializes only selected native key/value (K/V) detail. We ask whether these two forms of virtualization compose without conflating model-weight and semantic-memory policy. We audit AirLLM 3.3.0 at commit `3099999`, implement a capability-gated runtime provider, reuse the existing Hugging Face PRA attention path, and introduce a layer-local lifecycle adapter with HOT, layer-streamed, and hybrid PRA residency. On Apple M5, AirLLM's separate MLX path executes TinyLlama-1.1B from 2.20-GB layer shards. Reducing a 653-token prompt to 38 selected tokens preserved the 12-token answer prefix, but throughput remained about 0.13 token/s because weight streaming dominated. In a 119-condition controlled lifecycle study, layer-streaming reduced balanced-profile PRA peak residency from 0.50 to 0.125 MiB; independent prefetch was faster than sharing AirLLM's one-worker weight executor. On a 4-GiB GTX 950M, injected-but-disabled PRA preserved the exact AirLLM output, and native PRA encoded and consumed a 15-token reference through four layers. Native decode was 10.6% slower than selected-text E0, so this establishes mechanism correctness rather than a speed advantage. A selector-frozen 60-example natural-QA follow-up on an RTX 5060 materialized up to 384 native tokens while removing 92.4–95.1% of E0's visible selected-text prompt. Native/selected F1 is 0.112/0.099 on 2Wiki, 0.101/0.215 on HotpotQA, and 0.082/0.136 on QASPER; exact output parity is 0–10%. Completion is 10.3% faster on 2Wiki, statistically indistinguishable in this descriptive cohort on HotpotQA, and 26.1% slower on QASPER. The run is therefore transport and memory evidence, not semantic parity. A separate 15-example, four-token diagnostic measures native/selected TTFT ratios of 2.186–2.351 and ITL ratios of 1.109–1.173; warm reuse does not remove this request-path penalty. The evidence supports independent policy and exposes the central systems tradeoff: eliminating context residency is valuable only when its transfers do not deepen the weight-I/O bottleneck.

1 Qualification Snapshot

Engine question. Can AirLLM virtualize model weights while PRA independently virtualizes selected semantic context, and does native transport already beat selected text? The mechanism composes, but the measured cold native path does not yet improve latency.

Recommendation. Keep selected-text E0 as the deployment default. Treat native AirLLM/HF E2 as research-only until selector-frozen natural QA demonstrates parity and repeated resource reuse amortizes encoding and transfer. Do not infer E3 from `storage_managed`: AirLLM does not yet schedule PRA placement, sharing, eviction, and accounting as first-class physical state.

Primer. E0 renders selected evidence as ordinary prompt text. E1 carries logical PRA identity, typed records, routing, and fallback policy without native attention. E2 consumes detached selected native K/V directly at designated layers. E3 requires scheduler-owned physical lifecycle. AirLLM's weight streaming is orthogonal to this ladder: it can coexist with E0 or E2 without changing their meaning.

2 Introduction

Inference memory is not one pool. Model parameters, sequential K/V, selected external K/V, temporary activations, and runtime allocations have different lifetimes and reuse patterns. AirLLM [1] attacks parameter residency

Table 1: Standard PRA engine result block. Controlled accounting and live measurements are kept distinct; NOT MEASURED is explicit.

Field	Result
Validated / observed level	E0 validated on AirLLM/MLX; E2 mechanism observed on AirLLM/HF; E3 not observed
Model / hardware	TinyLlama-1.1B; Apple M5 16 GB, GTX 950M 4 GiB, and RTX 5060 8 GiB
Quality	Natural native/selected F1: 0.112/0.099 (2Wiki), 0.101/0.215 (Hotpot), 0.082/0.136 (QASPER)
Visible / native context	Smoke: 19 visible plus 15 native; natural: 20–32 visible plus 353–382 native
Cold native execution	32.03 s, 10.6% slower than selected-text E0; reference encoding 7.12 s
Peak allocation / residency	RTX natural E2 peak 161–162 MiB versus E0 169 MiB; 4× controlled PRA-residency reduction
TTFT / ITL / warm reuse	15-example diagnostic: E2/E0 TTFT 2.186–2.351; ITL 1.109–1.173
Evidence tier	Live mechanism smoke, 60-example natural cohort, 15-example timing diagnostic, and 119-condition controlled study

by building a model skeleton on the meta device and streaming checkpoint shards through forward hooks. PRA attacks context residency: a query selects bounded regions from typed resources, and designated decoder layers consume their native K/V without exposing the whole resource as sequential prompt text.

This combination is appealing but not automatic. The same storage device and host-to-device path may carry both layer weights and PRA detail. Keeping PRA K/V HOT consumes memory deliberately freed by AirLLM. Streaming PRA K/V saves that memory, but adds another demand stream. Moreover, replacing an HF attention module changes its parameter path even when all pretrained tensors are reused, which can invalidate AirLLM's shard loader.

We study one question:

> Can model-weight streaming and query-addressed semantic-context streaming be controlled independently while still composing at an attention layer?

The paper makes four scoped contributions. First, it records a source-grounded AirLLM capability audit. Second, it adds a conservative AirLLM provider to the shared PRA runtime: selected-text E0 is the default, while native E2 must be requested explicitly. Third, it implements the parameter-name and device bridge needed to reuse the HF-native PRA path, plus a layer-local K/V lifecycle adapter. Fourth, it reports live Apple/MLX selected-text evidence and controlled residency, prefetch, and memory-frontier measurements. We distinguish measured live results, controlled-model results, source-audit conclusions, and pending CUDA evidence.

3 Two Independent State Planes

Figure 1 separates the semantic-context plane from the model-weight plane. The systems meet only when a designated consumer layer is resident and its selected K/V is active.

We account for peak memory as

$$M_{\text{peak}} = M_{\text{weights,hot}} + M_{\text{localKV}} + M_{\text{PRA,hot}} + M_{\text{temporary}} + M_{\text{framework}}. \tag{1}$$

PRA WARM/COLD bytes are reported separately because they are retained physical state but not accelerator working-set bytes. This decomposition matters: once AirLLM removes full-model residency, local K/V may become the dominant term.

4 AirLLM Audit

We pin AirLLM 3.3.0 at commit `309999931a3cc5572d8880e7e0f18d408f3a067b`. The audit script records source files and line numbers rather than inferring behavior from package names. Table 2 summarizes the resulting contract.

The source audit found HF delegation in `airllm_base.py`, lines 904–907; streaming-hook installation in the same file, lines 763–764; and expert hooks near line 834. The Mac dispatcher enters a separate MLX class in

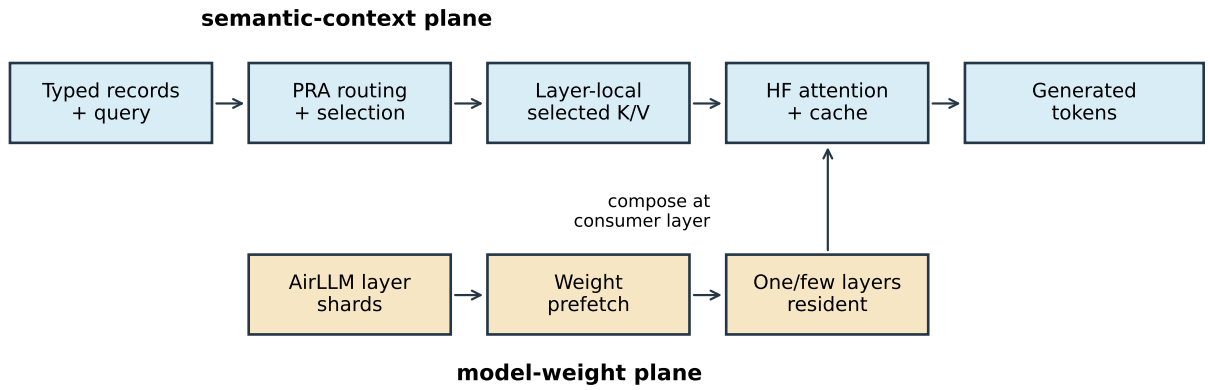

Figure 1: AirLLM owns weight-shard movement. The shared PRA runtime owns record identity, routing, authorization, and selected K/V. Their physical lifecycles compose at an HF attention consumer without sharing semantic policy.

Table 2: AirLLM mechanisms and PRA consequences at the pinned revision.

Mechanism	Source-audited behavior	PRA consequence
HF execution	`self.model.generate` and `self.model` own generation/forward	Reuse HF PRA on CUDA/Linux
Model skeleton	Built under Accelerate meta initialization	Request device cannot be inferred from embedding residency
Weight lifecycle	Module pre-hook loads; post-hook returns weights to meta	Attach PRA lifecycle after weight hook
Prefetch	One-worker thread executor	Shared scheduling can serialize two reads
Pinned host memory	Per-prefetched-layer bound	PRA pinning needs a separate explicit budget
Sparse experts	Routed expert pre/post hooks when topology permits	Expert and context selection remain independent
macOS path	Separate `AirLLMLlamaMlx` implementation	E0 only; do not claim HF-native PRA

`auto_model.py:7` and does not expose AirLLM's HF `.model`. Consequently, platform capability is not uniform: the Mac run validates AirLLM weight streaming and E0 selected text, not native PRA.

5 Integration

5.1 Capability-Gated Provider

The shared provider registry now includes `airllm`. It advertises E0 by default. Setting `native_hf_pra=true` enables E2 only on the HF-backed path. The `storage_managed` option changes how E2 tensors are retained; it does not promote the path to E3 because AirLLM does not own their complete scheduler lifecycle. The provider is embedded-only because AirLLM is a Python model wrapper rather than an HTTP scheduler. This prevents the CLI and product matrix from presenting the MLX path as native merely because the package imports.

5.2 Reusing HF Attention

PRA wraps selected `self_attn` modules while retaining each original attention module as `original_attention`. AirLLM shards still contain ordinary keys such as

```
model.layers.7.self_attn.q_proj.weight
```

The adapter maps that key, only for PRA-injected layers, to

```
model.layers.7.self_attn.original_attention.q_proj.weight
```

at placement time. Checkpoint files are neither rewritten nor duplicated. Unselected attention and all non-attention keys are unchanged. The request device is taken from AirLLM's `running_device`; relying on an embedding parameter would incorrectly return `meta` when embeddings are streamed.

In simplified form, the native bridge is:

```
pra = PRAForCausalLM.from_model(airllm.model, airllm.tokenizer)
for shard_key, tensor in layer_shard:
    target = insert_original_attention(shard_key) if pra_layer else shard_key
    airllm.place(target, tensor)
pra.request_device = airllm.running_device
```

AirLLM continues to invoke Transformers. PRA continues to use the model's own Q/K/V projections, RoPE implementation, and cache semantics.

5.3 Layer-Local PRA Lifecycle

The lifecycle adapter registers its hooks after AirLLM. PyTorch therefore runs:

```
AirLLM pre: load W_l
PRA pre:    activate selected (K_l, V_l)
forward:    local attention + selected-memory attention
AirLLM post: release W_l
PRA post:   release (K_l, V_l), unless policy keeps it HOT
```

The adapter supports three residency modes: HOT keeps every selected consumer layer resident; `layer_streamed` retains only the active layer; `hybrid` keeps an explicit layer subset HOT. Closing an adapter releases all active detail, enforcing the invariant that a later request cannot inherit another request's selected memory.

6 Experimental Method

6.1 Evidence Tiers

- Source audit: pinned source and runtime-import facts.
- Live engine E0: actual AirLLM/MLX model loading and generation.
- Controlled model: measured hooks, stores, transfers, and memory accounting with a deterministic eight-layer execution model; no language quality.
- Live native smoke: CUDA HF-backed mechanism and parity tests.

6.2 Live Apple Run

The live E0 run used an Apple M5 with 16 GB unified memory, Python 3.11, AirLLM 3.3.0, MLX 0.32.2, Transformers 5.12.1, and `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. AirLLM produced 50 shard/marker files totaling 2,200,159,150 bytes. We froze one evidence sentence and compared it as selected text against prompts containing repeated irrelevant archive notes. Greedy decode emitted 12 tokens.

6.3 Controlled Lifecycle Sweep

The controlled model has eight decoder layers, four K/V heads, 64-dimensional heads, FP16 K/V, 128 selected tokens, and 128 local query tokens. Each streamed weight layer is modeled as 8 MiB over an 800-MiB/s store with 3 ms of layer compute. We crossed backing contexts of 2K, 8K, 32K, and 64K tokens; three consumer profiles; three residency modes; and three prefetch policies, then added WARM, COLD, and SOURCE restoration controls. The result contains 119 rows with three repeats for timed native conditions.

Table 3: Live AirLLM/MLX E0 generation. One run per condition; 12 greedy output tokens. TTFT is first yielded token, not HTTP serving TTFT.

Condition	Input tok.	TTFT (s)	Total (s)	tok/s	Output prefix
Full, short	185	7.65	89.18	0.135	`The project access code is CYAN-ORBIT-`
Selected	38	7.60	93.00	0.129	same
Full, longer	653	8.27	90.23	0.133	same
Selected	38	7.99	88.97	0.135	same

Table 4: Live AirLLM/HF execution on a 4-GiB GTX 950M. Four greedy output tokens; peak is PyTorch CUDA allocation, not total device use.

Condition	Visible tok.	Native tok.	Total (s)	Peak (MiB)
Full-context E0	377	0	40.29	160.42
Selected-text E0	38	0	28.95	135.20
PRA injected, disabled	38	0	31.35	135.20
Native PRA	19	15	32.03	135.63

7 Results

7.1 Selected Text on AirLLM/MLX

Selected text reduced visible input by 79.5% relative to the 185-token prompt and by 94.2% relative to the 653-token prompt. It preserved the complete 12-token output prefix in all four conditions. The run stopped one generated token before the final digits of the synthetic code, so we report prefix agreement rather than exact-answer accuracy. Cold initialization, including download/sharding, took 109.6 s; reuse of existing shards initialized in 0.57 s. The first instrumented run observed about 1.10 GiB maximum change in system available memory, while process RSS grew from 346 to 832 MiB. System-available memory is noisy on a shared host and is not treated as process peak memory.

The small TTFT and throughput differences are the main result, not a failure of selection. Each decoded token streams the model layers again. At these prompt lengths, reducing sequential context cannot remove that fixed weight-I/O floor. This finding motivates native persistent K/V chiefly for repeated queries and longer backing contexts, not as a universal latency improvement.

The same run also exposed an AirLLM/MLX compatibility boundary: SmolLM2-135M uses tied embeddings and had no independent `lm_head` shard, but the MLX generation path attempted to load one. We retain this blocked run as an engine/model-family finding and do not count it as PRA evidence.

7.2 HF-Backed CUDA Correctness

The full and selected E0 conditions generated the same four-token prefix. Selected text removed 89.9% of visible input, reduced completion latency by 28.2%, and reduced peak allocated CUDA memory by 15.7%. After PRA injection, the disabled condition matched the selected E0 token sequence exactly. This is the key parameter-placement result: every streamed decoder shard reached the PRA-wrapped path without changing disabled semantics.

Reference encoding took 7.12 s for 15 tokens. Native generation exposed only 19 query tokens sequentially and materialized all 15 selected K/V tokens at the configured final four consumer layers. It generated the same prefix in 32.03 s and peaked at 135.63 MiB. Relative to selected-text E0, this is a 10.6% latency cost and 0.3% peak-allocation increase in a cold one-query smoke. Persistent reuse is therefore the economic gate: the one-time 7.12-s encoding cost must be amortized across repeated queries before native PRA can improve this frontier.

7.3 Selector-Frozen Natural Follow-Up

We next froze 20 routed selections from each of QASPER, HotpotQA, and 2WikiMultiHopQA. Selected-text E0 and the native candidate received the same evidence; the native path consumed 353–382 selected tokens on average through the final four layers. Each condition used one cold generation and one warm repeat, for 300 requests including the full-context control. Table 5 and Figure 2 are generated from the raw JSON artifact.

Table 5: Natural AirLLM/HF follow-up on the RTX 5060. Completion is native candidate divided by selected E0; lower is faster. Pair exact compares complete greedy output strings. TTFT is explicit missing data from the legacy runner.

Dataset	State	E0 F1	E2 F1	Pair exact	Completion	TTFT	Visible ↓
2Wiki	cold	0.099	0.112	0.100	0.897	–	93.2%
2Wiki	warm	0.099	0.112	0.100	0.894	–	93.2%
HotpotQA	cold	0.215	0.101	0.050	0.997	–	92.4%
HotpotQA	warm	0.215	0.101	0.050	0.999	–	92.4%
QASPER	cold	0.136	0.082	0.000	1.261	–	95.1%
QASPER	warm	0.136	0.082	0.000	1.238	–	95.1%

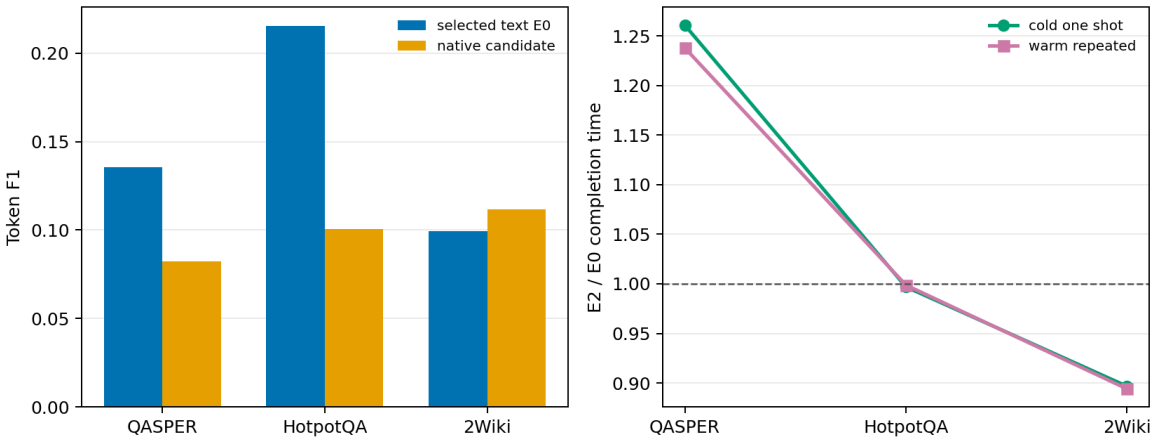

Figure 2: Matched quality and completion cost. The same selected evidence is used in both representations; the native candidate is not semantically equivalent to selected text on this partial-consumer configuration.

Native execution removed 92.4–95.1% of the visible selected-text prompt and reduced peak allocated CUDA memory from about 169 MiB to 161–162 MiB. Completion was 10.3% faster on 2Wiki, effectively unchanged on HotpotQA, and 26.1% slower on QASPER. Cold reference encoding averaged 10.91 s and must be amortized separately. These are positive bounded-context and modest residency results, but not a general latency win.

Quality is the decisive open gate. Native F1 slightly exceeded selected text on 2Wiki (0.112 versus 0.099), but fell from 0.215 to 0.101 on HotpotQA and from 0.136 to 0.082 on QASPER. Exact output parity was 10%, 5%, and 0%, respectively. The four-layer consumer is therefore not execution-equivalent to placing the same evidence in every ordinary decoder layer. The mixed direction also prevents interpreting divergence alone as uniform damage or benefit. Native execution remains research-only until a capable larger model and a geometry-matched full-consumer condition close this semantic gap.

7.4 Direct Token-Timing Diagnostic

The preceding 60-example runner predates token callbacks and therefore cannot support a TTFT or ITL claim. We separately reran the first five examples from each dataset with four generated tokens and an HF streamer timestamp on every decoded token. This 15-example diagnostic freezes the same selected evidence across E0 and E2, excludes one-time reference encoding from request latency, and is used only for transport timing; its short outputs do not replace the larger cohort's quality measurements.

Native E2 more than doubled mean TTFT in every dataset: the E2/E0 ratio ranged from 2.186 to 2.351. ITL rose by 10.9–17.3%, and complete four-token requests rose by 10.3–16.9%. Warm repeats were nearly identical to cold requests even though the reference had already been encoded. The persistent TTFT gap is therefore on the native request/consumer path rather than in the separately measured 10.88-s one-time reference encoding. The present trace does not attribute that gap to a single operation; layer streaming, selected-K/V materialization, and the four direct-plus-memory attention consumers remain combined. Profiling and overlap are required before claiming that native PRA amortizes well under AirLLM.

Table 6: Directly measured AirLLM/HF timing on the RTX 5060. Times are milliseconds; each cell averages five requests. Ratios are E2 divided by E0, so values above one favor selected-text E0. Completion is the complete four-token request ratio.

Dataset	State	E0 TTFT	E2 TTFT	Ratio	E0 ITL	E2 ITL	Ratio	Completion
2Wiki	cold	5167.4	11295.6	2.186	5041.0	5736.0	1.138	1.123
2Wiki	warm	5014.9	11209.0	2.235	5148.0	5710.8	1.109	1.103
HotpotQA	cold	4905.6	11195.5	2.282	4953.3	5700.2	1.151	1.149
HotpotQA	warm	4913.2	11280.2	2.296	4931.3	5695.8	1.155	1.150
QASPER	cold	4750.6	10955.4	2.306	4814.3	5645.4	1.173	1.169
QASPER	warm	4793.6	11271.4	2.351	4823.3	5628.2	1.167	1.169

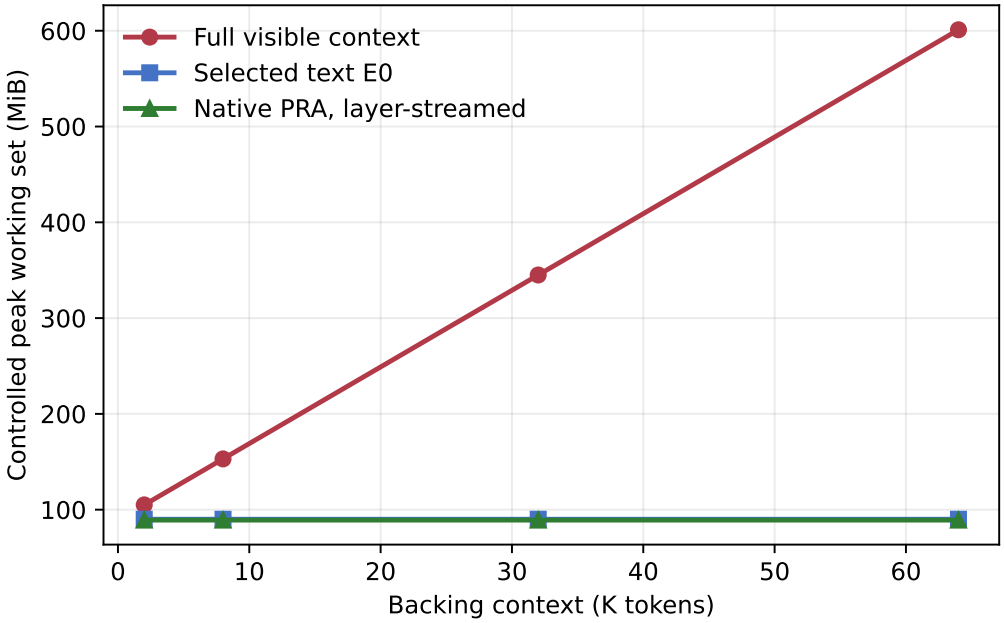

Figure 3: Controlled working-set decomposition. Full-context local K/V grows with backing length; selected text and native selected K/V remain bounded. These values are model-derived accounting, not GPU telemetry.

7.5 Residency Frontier

At 32K backing tokens under the balanced four-consumer profile, HOT PRA retained 0.50 MiB, hybrid retained 0.375 MiB, and layer-streamed PRA retained 0.125 MiB. Thus layer streaming reduced peak PRA detail by 4× relative to HOT for the same 128 selected tokens. In the controlled decomposition at 64K, full-context execution reached 601.0 MiB while balanced layer-streamed native PRA reached 89.125 MiB. The 85.2% reduction follows directly from holding selected evidence and query length fixed; it is a memory-accounting result, not evidence that 64K model quality is preserved.

7.6 Prefetch Interaction

For the 32K balanced hybrid condition, no PRA prefetch took 155.83 ms per controlled pass. Independent one-worker PRA prefetch took 133.25 ms, a 14.5% reduction. Coordinated prefetch on AirLLM's weight executor took 142.56 ms, 8.5% below no prefetch but 7.0% above independent prefetch. "Coordinated" is therefore not synonymous with "faster": a shared serial queue can prevent thread multiplication yet lose useful overlap. A production controller should choose between independent I/O and queue coordination from observed storage parallelism rather than platform labels.

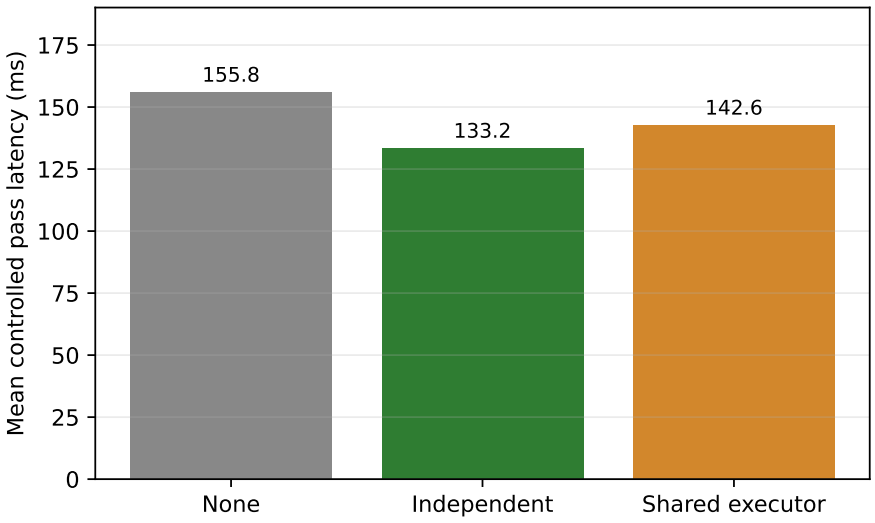

Figure 4: Controlled 32K/hybrid balanced-profile latency. Sharing AirLLM's one-worker executor avoids another thread but queues PRA behind weight reads.

Table 7: Evidence status at this revision.

Claim	Status	Evidence boundary
AirLLM delegates CUDA forward/generate to HF	Established	Pinned source audit
AirLLM/MLX runs layer-streamed TinyLlama	Established	Live M5 E0 run
Selected text preserves tested answer prefix	Established	Four live conditions, one example
PRA K/V can follow a layer-local lifecycle	Established	Tests plus 119-row controlled sweep
Native HF parameter placement bridge	Established	Unit tests and live disabled parity
Native AirLLM/PRA mechanism	Established	Live CUDA smoke plus 60 natural examples
Natural E0/E2 semantic parity	Open	0–10% exact pairs; material F1 losses on HotpotQA/QASPER
Large-model-beyond-VRAM advantage	Open	Requires a model larger than accelerator memory

8 What the Results Do and Do Not Establish

The measurements establish a clean ownership boundary and a feasible physical lifecycle. They show that a bounded selected context does not inherit backing context's K/V growth in the accounting model. They also show that AirLLM's weight-streaming cost can dominate short-prompt E0 inference, and that using one executor for two I/O streams may be inferior to bounded independent prefetch.

They do not establish that native PRA improves answer quality or TTFT for a model that cannot fit on the test accelerator. The 60-example follow-up shows lower allocated CUDA peak and workload-dependent completion time, but the HotpotQA and QASPER F1 losses keep semantic parity open. The CUDA smoke closes AirLLM disabled, injected-but-disabled, reference encoding, and native consumption. Direct four-token timing additionally finds a 2.186–2.351× TTFT penalty and a 1.109–1.173× ITL penalty for native E2. A scale claim still requires a conventional HF baseline, HOT versus layer-streamed native PRA, matched source/query geometry, and a model whose weights actually exceed accelerator memory.

9 Product Contract

The current product row is deliberately conservative:

Engine	Platform	Default	Native candidate	Evidence tier
AirLLM	macOS/MLX	E0 selected text	unavailable	live smoke
AirLLM	CUDA/HF	E0 selected text	opt-in E2	natural transport candidate

The runtime exposes `airllm` only as an embedded provider. Native flags must be explicit. No `QUALITY_MAX` product claim follows from this small calibration; profiles remain candidates until natural-task validation.

10 Limitations and Next Experiments

The M5 cohort is one synthetic retrieval example and one Llama-family model. Its MLX path differs architecturally from AirLLM's HF path. The controlled model uses explicit bandwidth and compute parameters and cannot estimate kernel-level attention cost or language quality. The host was under unrelated memory load, so available-memory deltas are supporting diagnostics only.

The next decisive experiment is not another hook microbenchmark. The RTX cohort should be repeated with a capable model and both a geometry-matched full-consumer and an economical late-consumer E2. It should then extend to multi-query same-resource and shared-resource reuse while reporting gold answer log probability, TTFT, inter-token latency, memory decomposition, and cache hits. The timing trace should split layer loading, selected-K/V transfer, materialization, direct attention, and memory attention to localize the first-token penalty. A model materially larger than VRAM is required to test the claimed weight-streaming regime. A sparse-MoE run is useful only after dense-model correctness; expert selection and PRA routing must remain separately attributed.

11 Related Work

AirLLM [1] belongs to layer/expert streaming and offload systems. FlexGen [3] jointly schedules parameters, activations, and K/V across GPU, CPU, and disk for throughput-oriented constrained inference. Transformers provides the architecture and cache semantics reused by the integration described here [2]. PagedAttention and vLLM [4] address fragmentation and sharing of sequential serving K/V; their scheduler-visible block abstraction differs from AirLLM's layer-streamed parameter lifecycle. Retrieval and prompt compression move selected text into the ordinary prefix. PRA instead studies query-selected non-prefix native state. The distinguishing combination here is parameter streaming plus semantic K/V selection, with the two policies remaining independently measurable.

12 Conclusion

AirLLM and PRA target different memory terms and can share an HF execution path without sharing policy. The integration requires more than calling two APIs: PRA-wrapped parameter names must be reconciled with AirLLM shards, request device identity cannot be read from meta-resident embeddings, teardown must prevent selected-detail leakage, and prefetch queues must be measured rather than assumed. Live MLX E0 results show substantial visible-token reduction but also expose weight streaming as the dominant latency floor. Controlled results show a 4× PRA peak-residency reduction from layer-local streaming and a prefetch tradeoff between independent overlap and shared-queue discipline. A live CUDA smoke closes parameter-placement, disabled-parity, reference-encoding, and native-consumption gates. The 60-example natural follow-up removes 92.4–95.1% of visible selected-text tokens and lowers allocated CUDA peak, but completion cost depends on the workload and HotpotQA/QASPER lose substantial F1. Direct timing also finds a persistent 2.19–2.35× TTFT ratio under the current four-consumer path. Semantic parity and an execution advantage therefore remain open. The remaining scientific gate is a geometry-matched full-consumer evaluation with a capable model at a scale where both weight and context virtualization are necessary.

Reproducibility

Scripts are in `experiments/paper6_6_airllm`. Raw JSON, figures, and the source capability audit are under `docs/papers/shared/results/paper6_6_airllm` and the paper's `figures` directory. Unit tests cover provider negotiation, selective parameter-key remapping, lifecycle ordering, HOT reuse, teardown, and memory decomposition. Every live artifact records package versions, hardware, model, and its evidence tier.

References

[1] G. Lyongavin et al. *AirLLM: Run large language models on low-end GPUs*. Pinned source repository, 2026. `https://github.com/lyogavin/airllm`.

[2] T. Wolf et al. Transformers: State-of-the-art natural language processing. *EMNLP: System Demonstrations*, 2020.

[3] Y. Sheng et al. FlexGen: High-throughput generative inference of large language models with a single GPU. *ICML*, 2023.

[4] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.